

Heart-Rate & HRV Forecasting System

Datasets: Dataset3 (Stress-Predict) + WESAD (both Empatica E4)

Goal

Predict a person's **Heart Rate (HR)** and **Heart-Rate Variability (HRV / RMSSD)** a few minutes into the future, from a wrist smartwatch.

Two datasets that share the exact same sensors are combined so one model can learn from both:

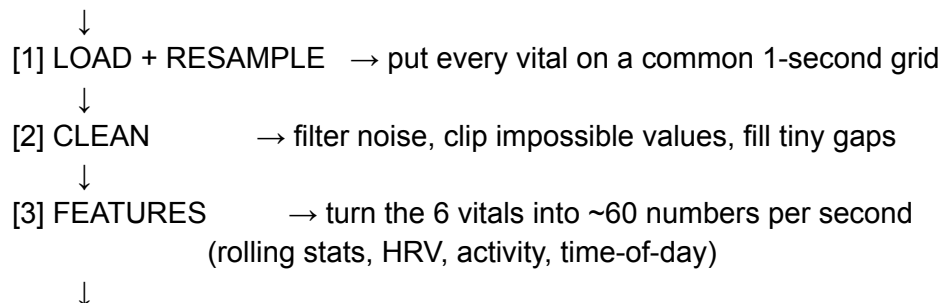
- **Dataset3** = Stress-Predict (35 healthy adults, Empatica E4)
- **WESAD** (15 healthy subjects, Empatica E4)

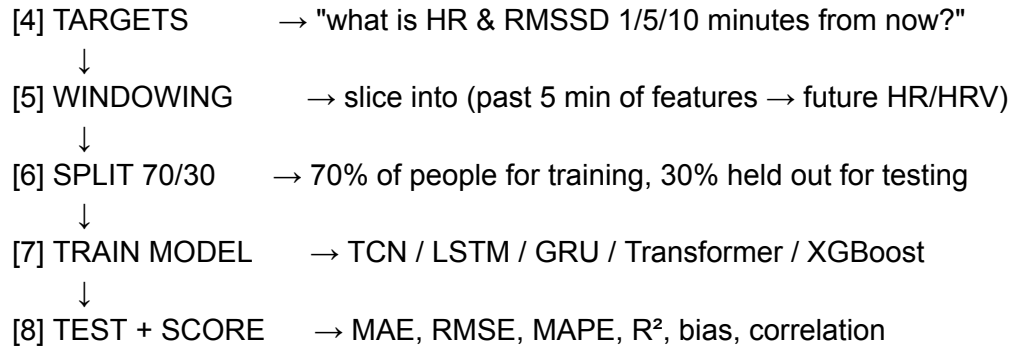
Both are Empatica E4 exports with the same six vitals. Only these are used — nothing else:

#	Signal	Description	Sample Rate
1	HR	Heart Rate (bpm)	1 Hz
2	IBI	Inter-Beat Interval (s) — used for HRV	per heartbeat
3	ACC	3-axis Accelerometer (motion)	32 Hz
4	BVP	Blood Volume Pulse (raw pulse)	64 Hz
5	EDA	Electrodermal Activity (μ S)	4 Hz
6	TEMP	Skin Temperature ($^{\circ}$ C)	4 Hz

Pipeline Overview

RAW E4 FILES (HR, IBI, ACC, BVP, EDA, TEMP)





Everything is causal: at any second we only use information from the past, never the future. This is what lets the same code run live on a real watch stream.

Step-by-Step Detail

Step 1 — Load + Resample

src/data/loader.py, sync.py

The six vitals are recorded at different speeds, so they are aligned on one common 1-second clock:

- **HR** → average per second
 - **ACC** → remove gravity, compute motion magnitude, take mean/std/max per second
 - **EDA** → low-pass smooth, then average per second
 - **TEMP** → average per second
 - **BVP** → band-pass filtered, kept at full rate for pulse analysis
 - **IBI** → the real beat-to-beat gaps, used to compute HRV
-

Step 2 — Clean

src/data/cleaning.py

- HR clipped to human range (30–220 bpm)
 - IBI clipped to plausible beat gaps (0.3–2.0 s)
 - BVP band-pass filtered (0.5–8 Hz) to isolate the real pulse
 - EDA low-pass filtered to remove jitter
 - ACC high-pass filtered to remove gravity (measures movement, not tilt)
 - Gaps ≤ 5 s are interpolated; longer gaps stay "missing" and any window touching them is skipped — no training on invented data
-

Step 3 — Features

`src/features/*.py`

The six vitals are converted into ~60 numbers per second describing the recent past. All windows look backward only (causal).

Feature Family	What's Computed
HR	Rolling mean/std/min/max over 30/60/300 s; short-term trend; rate-of-change; distance from recent baseline
IBI	HRV metrics RMSSD and SDNN over 60/300 s windows. Low HRV = stress/fatigue; high HRV = relaxed/recovered
ACC	Movement intensity + activity bucket (rest / light / moderate / vigorous)
BVP	Pulse-shape statistics from the optical waveform
EDA	Tonic level + phasic "arousal" bursts (sweat responses)
TEMP	Skin-temperature level and deviation from its own baseline
Time	Hour-of-day as sine/cosine + a personal per-hour HR baseline ("circadian" features)

Step 4 — Targets

`build_features.py`

For every second t , the following future values are created as prediction targets:

- **HR_target_60s** = HR at $t+60$ (1 minute ahead)
- **HR_target_300s** = HR at $t+300$ (5 minutes ahead)
- **HR_target_600s** = HR at $t+600$ (10 minutes ahead)

The same three horizons are created for RMSSD (HRV forecasting):

- **RMSSD_target_60s / _300s / _600s**

These are future readings shifted back to "now" so the model learns: *past features* → *future HR & HRV*.

Step 5 — Windowing

`src/models/datasets.py`

A model needs fixed-size inputs, so a window is slid over time:

- **X** = the last 300 seconds (5 min) of all features → shape (300, ~60)
- **y** = the HR & RMSSD targets at the end of the window

Any window containing a missing value is skipped.

Step 6 — Train / Test Split (70 / 30)

`multi_dataset_pipeline.py`

Subjects from both datasets are pooled and split **by person**:

- **70% of subjects** (from each dataset) → **Train**
- **30% of subjects** (held out entirely) → **Test**

Why split by person, not randomly? Time-series points next to each other are nearly identical, so random splitting would put the same person in both train and test — that's cheating. Holding out whole people tests the real question: *does it work on a new person it has never seen?*

Features are standardized (mean 0, std 1) using train statistics only, then the same transform is applied to test subjects. Because both datasets share the same six vitals, the model uses the full feature set.

Step 7 — The Models

`src/models/*.py`

Five models are supported for benchmarking. Each outputs, for every horizon, both a **predicted value** and an **uncertainty** (mean + std), trained with Gaussian negative-log-likelihood. The std provides confidence intervals.

Model	Type	Notes
TCN (<i>default, usually best</i>)	Temporal Convolutional Network	Stacked dilated 1-D convolutions; sees long history efficiently

LSTM	Recurrent network	Strong baseline with memory gates
GRU	Recurrent network	Lighter LSTM; often equally good
Transformer	Attention-based	Self-attention over the window
XGBoost	Gradient-boosted trees	No memory; fast, robust sanity-check baseline

Step 8 — Training

src/models/train.py

- **Deep models:** Adam optimizer, 30 epochs by default; best validation checkpoint is kept (prevents overfitting). Loss rewards an accurate value and a well-calibrated uncertainty.
 - **XGBoost:** Builds 200 trees, each correcting the previous trees' errors.
-

Step 9 — Testing & Scoring

src/evaluation/metrics.py

Scored only on the held-out 30% of people, per horizon and overall, and broken down per dataset (Stress-Predict vs WESAD):

Metric	Meaning	Direction
MAE	Mean Absolute Error (avg bpm off)	Lower = better
RMSE	Root Mean Squared Error (punishes big misses)	Lower = better
MAPE	Mean Absolute % Error	Lower = better
R²	Fraction of HR variation explained (1 = perfect, 0 = guessing mean)	Higher = better
Bias	Average signed error (+ over-predict, – under-predict)	Closer to 0 = better
Pearson r	Correlation of predicted vs true HR shape	Higher = better

Also reported: std_error, median_AE, max_AE (for spread and worst-case analysis).

How to Run

Set dataset paths in `configs/config_multi.yaml`:

- `stress_predict.raw_root` → your Dataset3/Raw_data path
- `wesad.raw_root` → your WESAD path

Pooled 70/30 across both datasets (main run):

```
python3 scripts/run_multi_dataset.py --model tcn
python3 scripts/run_multi_dataset.py --model xgboost
```

Train on a single dataset only:

Dataset3

```
python3 scripts/run_training.py --subject S01 --model tcn --config configs/config.yaml
```

WESAD

```
python3 scripts/run_training.py --subject S2 --model tcn --config configs/config_wesad.yaml
```

Outputs:

- `models/multi/<model>/` — pooled model + scaler + metadata
- `models/<SID>/<model>/` — single-subject models
- `processed/` — feature CSVs

Q&A

Q1. What exactly do we predict?

Heart rate and HRV (RMSSD) 1, 5, and 10 minutes into the future, using only the past readings of the six E4 vitals.

Q2. Which vitals do we use, and why only these?

HR, IBI, ACC, BVP, EDA, TEMP. These are present in both Dataset3 and WESAD. Using only shared vitals means one model can train on both datasets without missing-data problems.

Q3. Why combine two datasets?

More people and more variety = a model that generalises better. Because both are Empatica E4, their signals are directly compatible.

Q4. Why split 70/30 by person instead of randomly?

Time-series points are highly correlated; random splitting leaks the same person into train and test, inflating the score. Holding out whole people tests true generalisation to someone new.

Q5. What is "causal" and why does it matter?

Every feature uses only past data. Without this, offline scores would be optimistic and the model would fail when run live on a real watch.

Q6. What is HRV / RMSSD?

Beat-to-beat variation in the heartbeat, computed from IBI. High = relaxed/recovered; low = stress/fatigue. We both use it as a feature and forecast it.

Q7. Why five models?

To benchmark. TCN usually wins, but if the simple XGBoost baseline matches a deep model, the extra complexity isn't worth it.

Q8. Why predict an uncertainty (std), not just a number?

A range (mean $\pm$ std) gives confidence intervals and tells us when the model itself is unsure during noisy or unusual periods.

Q9. What do MAE, RMSE, R^2 , and bias mean?

MAE = average error in bpm. RMSE = error that punishes big misses. R^2 = how much of HR's variation we explain (1 = perfect). Bias = systematic over/under-shoot. Together they cover accuracy, worst case, explanatory power, and offset.

Q10. How is this real-time capable?

Causal, cheap features + small models + one forward pass per second. The same code that scores offline can run on a live stream.

Q11. What are the limitations?

Wrist PPG is noisier than clinical ECG, so HRV is approximate. Both cohorts are limited in size and diversity, so results may not transfer to every population.